

Extended Entity Relationship (EER) Model

- ER model has been widely used but have some shortcomings.
- Difficult to represent cases where an entity may have varying attributes dependent upon some property.
- ER model has been extended into Extended Entity Relationship model which includes more semantics such as generalization, categorization and aggregation.

Cardinality: One-to-one relationship

A one-to-one relationship between set A and set B is defined as: For all a in A, there exists at most one b in B such that a and b are related, and vice versa.

Example

A president leads a nation.

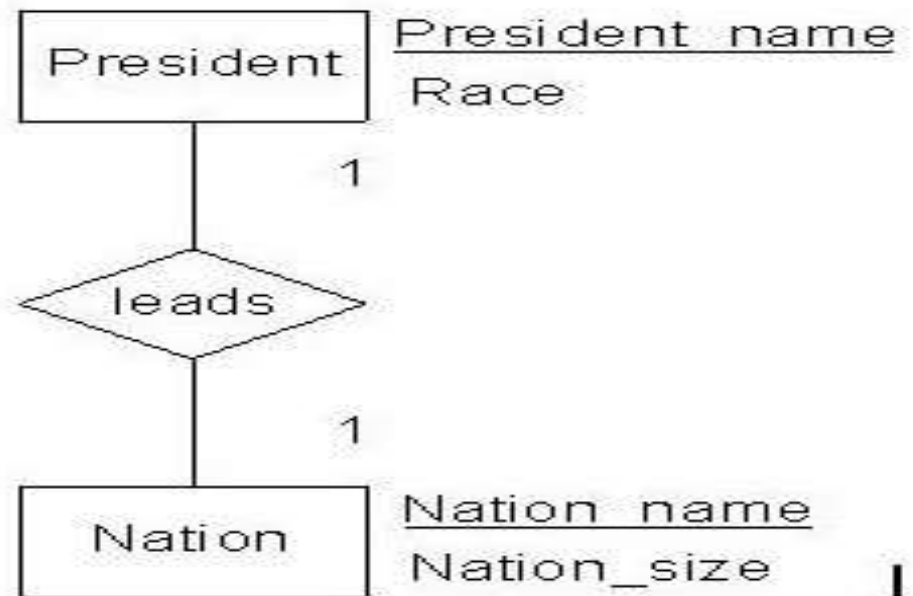
Relational Model:

Relation President (President_name, Race, *Nation_name)

Relation Nation (Nation_name, Nation_size)

Where underlined are primary keys and "*" prefixed are foreign keys

Extended Entity Relationship mod



Cardinality: Many-to-one relationship

A many-to-one relationship from set A to set B is defined as: For all a in A, there exists at most one b in B such that a and b are related, and for all b in B, there exists zero or more a in A such that a and b are related.

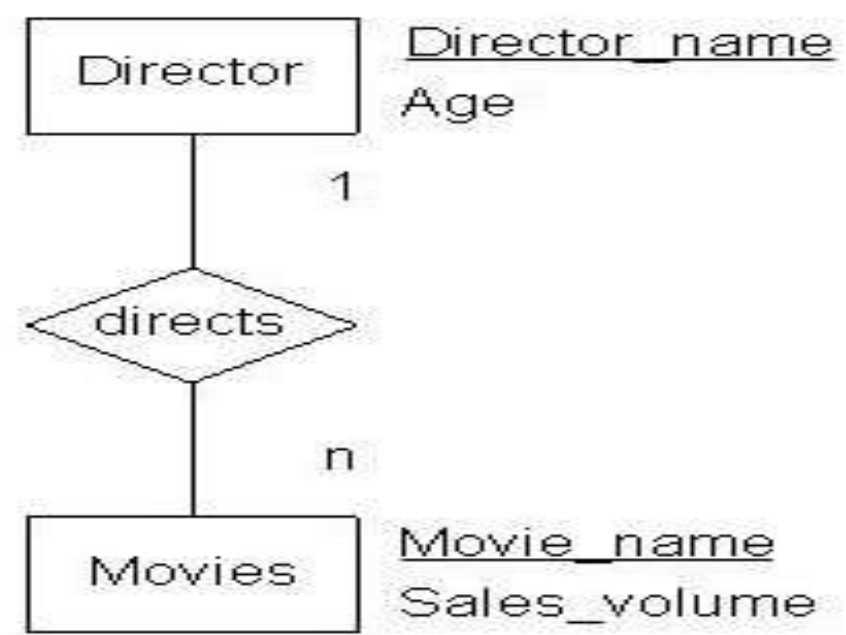
Example A director directs many movies.

Relational Model:

Relation Director (Director_name, Age)

Relation Movies (Movie_name, Sales_volume, *Director_name)

Extended entity relationship model:



Cardinality: Many-to-many relationship

A many-to-many relationship between set A and set B is defined as: For all a in A, there exists zero or more b in B such that a and b are related, and vice versa.

Example

Many students take many courses such that a student can take many courses and a course can be taken by many students.

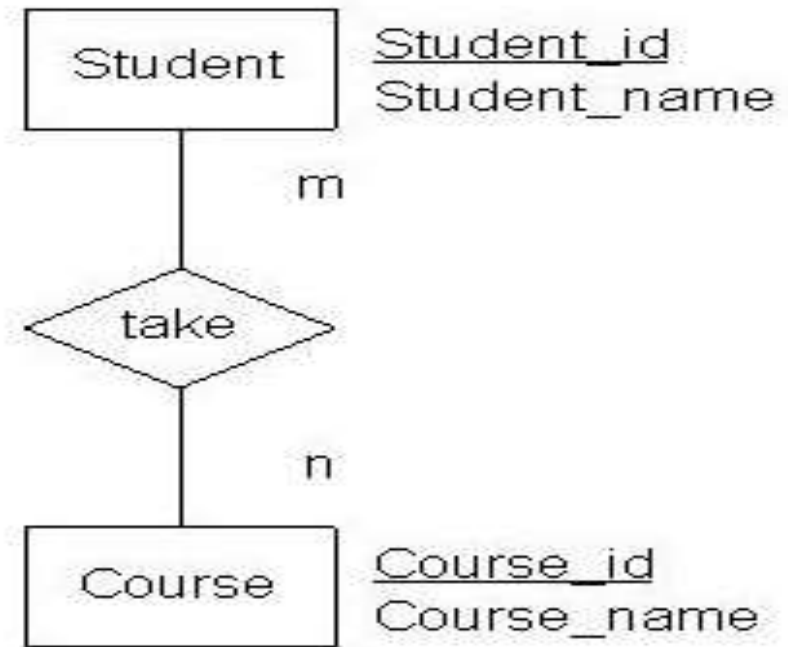
Relational Model:

Relation Student (Student_id, Student_name)

Relation Course (Course_id, Course_name)

Relation take (*Student_id, *Course_id)

Extended entity relationship model:



Data Semantic: Is-a (Subtype) relationship

The relationship A isa B is defined as: A is a special kind of B.

Example

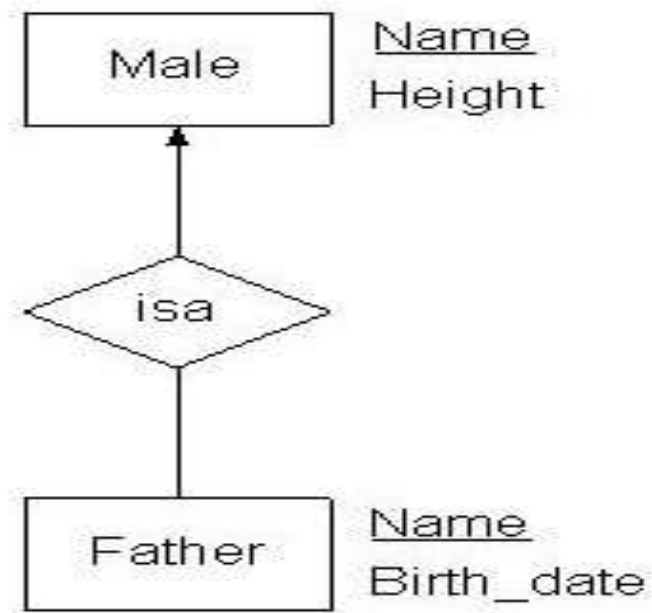
Father is Male.

Relational Model:

Relation Male (Name, Height)

Relation Father (*Name, Birth_date)

Extended entity relationship model



Data Semantic: Disjoint Generalization

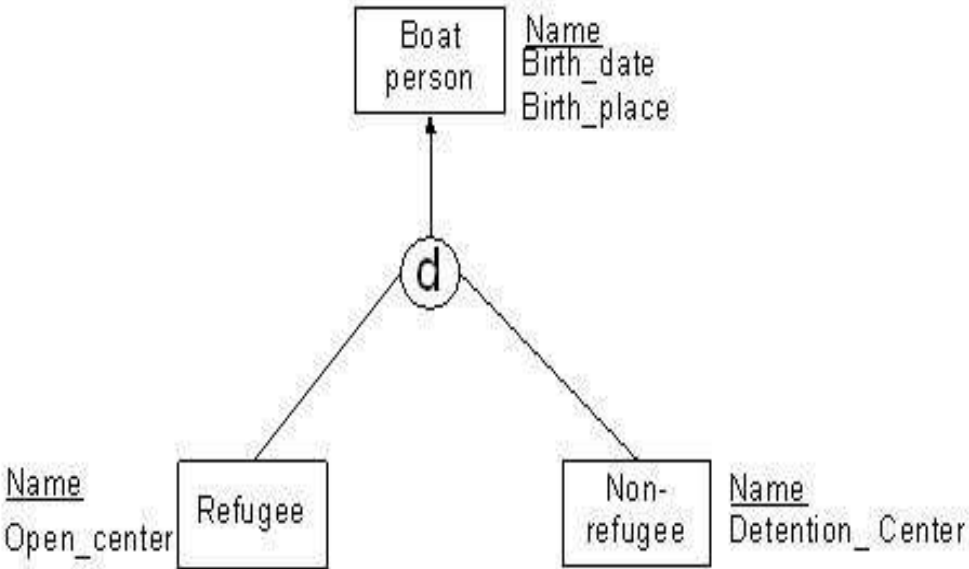
- Generalization is to classify similar entities into a single entity. More than one is-a relationship can form data abstraction (i.e. super-class and subclasses) among entities.
- A subclass entity is a subset of its super-class entity. There are two kinds of generalization.
- The first is disjoint generalization such that subclass entities are mutually exclusive.
- The second is overlap generalization such that subclass entities can overlap each other.

Example of Disjoint Generalization

A refugee and a non-refugee can both be a boat person, but a refugee cannot be a non-refugee, and vice versa.

Relational Model:

- Relation Boat_person (Name, Birth_date, Birth_place)
- Relation Refugee (*Name, Open_center)
- Relation Non-refugee (*Name, Detention_center)



Extended entity relationship model:

Data Semantic: Overlap Generalization

Example of Overlap Generalization

A computer programmer and a system analyst can both be a computer professional, and a computer programmer can also be a system analyst, and vice versa.

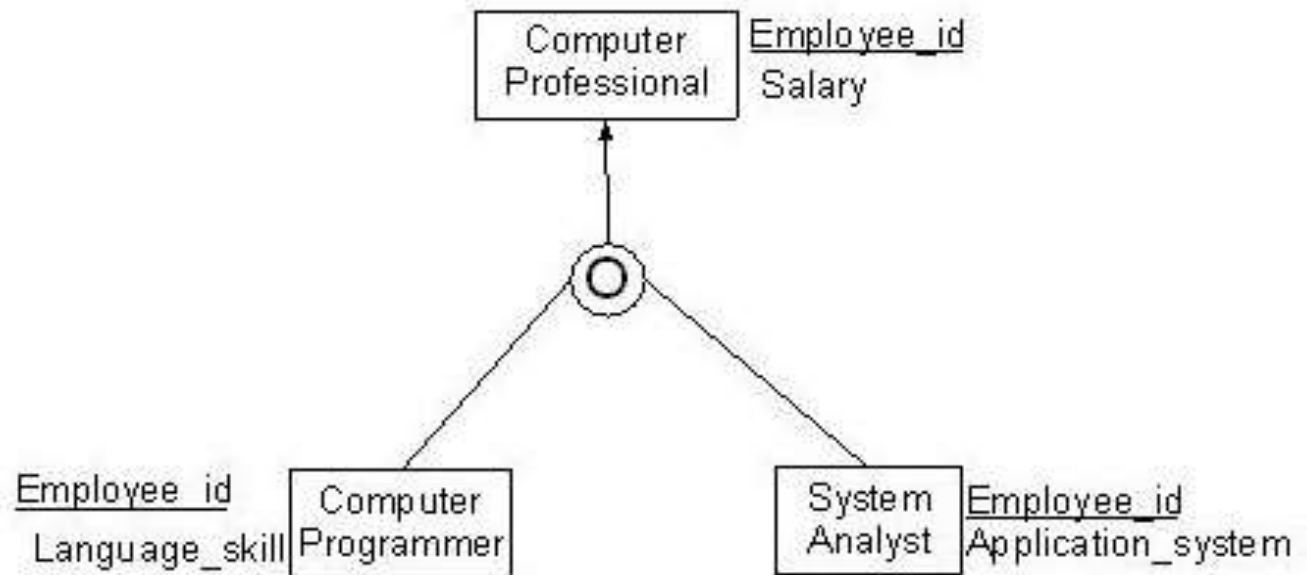
Relational Model:

Relation Computer_professional (Employee_id, Salary)

Relation Computer_programmer (*Employee_id, Language_skill)

Relation System_analyst (*Employee_id, Application_system)

Extended entity relationship model:



Data Semantic: Categorization Relationship

In cases the need arises for modeling a single super-class/subclass relationship with more than one super-class (es), where the super-classes represent different entity types. In this case, we call the subclass a category.

Relational Model:

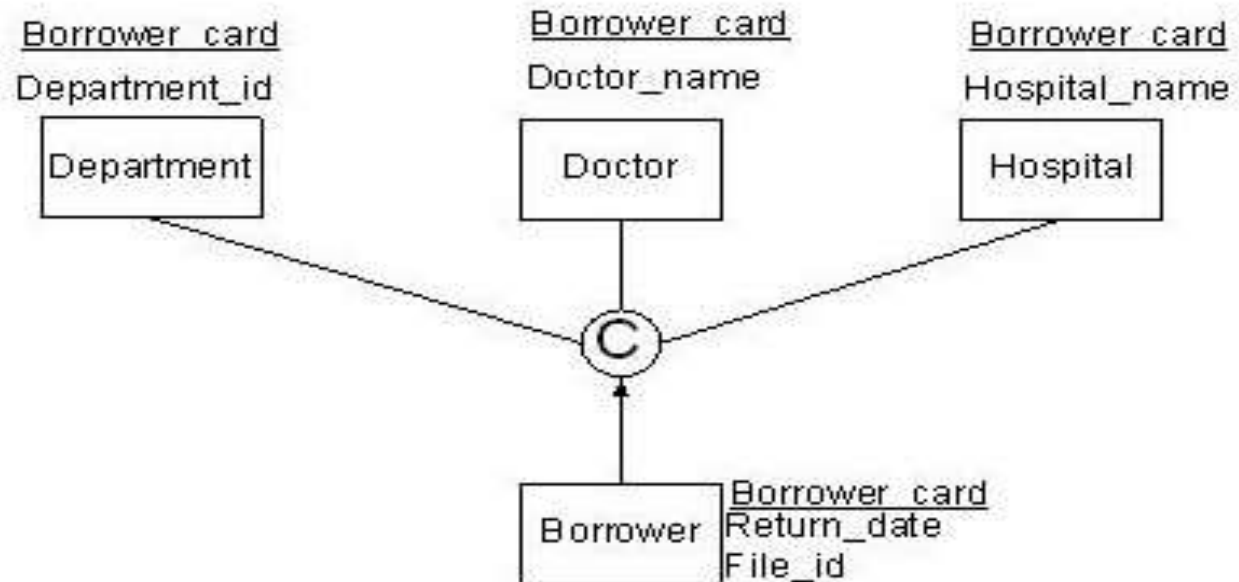
Relation Department (Borrower_card, Department_id)

Relation Doctor (Borrower_card, Doctor_name)

Relation Hospital (Borrower_card, Hospital_name)

Relation Borrower (*Borrower_card, Return_date, File_id)

Extended Entity Relationship Model



Data Semantic: Aggregation Relationship

Aggregation is a method to form a composite object from its components. It aggregates attribute values of an entity to form a whole entity.

Example

The process of a student taking a course can form a composite entity (aggregation) that may be graded by an instructor if the student completes the course.

Relational Model:

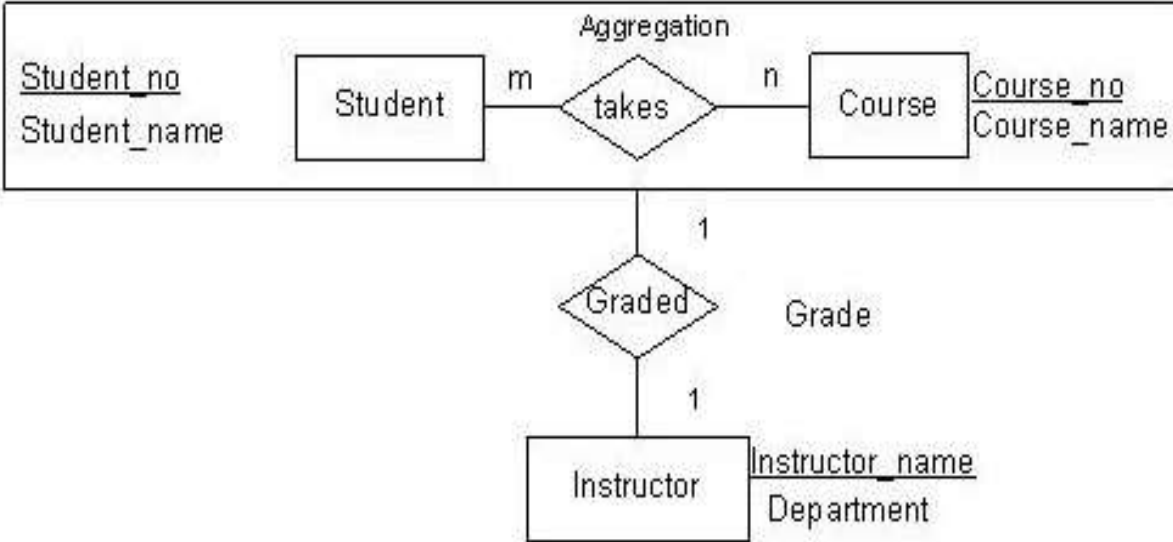
Relation Student (Student_no, Student_name)

Relation Course (Course_no, Course_name)

Relation Takes (*Student_no, *Course_no, *Instructor_name)

Relation Instructor (Instructor_name, Department)

Extended Entity Relationship Model



Data Semantic: Total Participation

An entity is in total participation with another entity provided that all data occurrences of the entity must participate in a relationship with the other entity.

Example

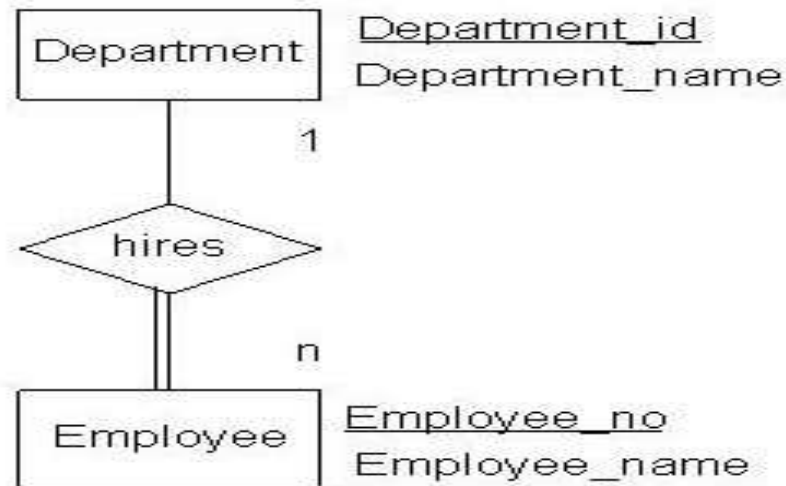
An employee must be hired by a department.

Relational Model:

Relation Department (Department_id, Department_name)

Relation Employee (Employee_id, Employee_name, *Department_id)

Extended entity relationship model:



Data Semantic: Partial Participation

An entity is in partial participation with another entity provided that the data occurrences of the entity are not totally participate in a relationship with the other entity.

Example

An employee may be hired by a department.

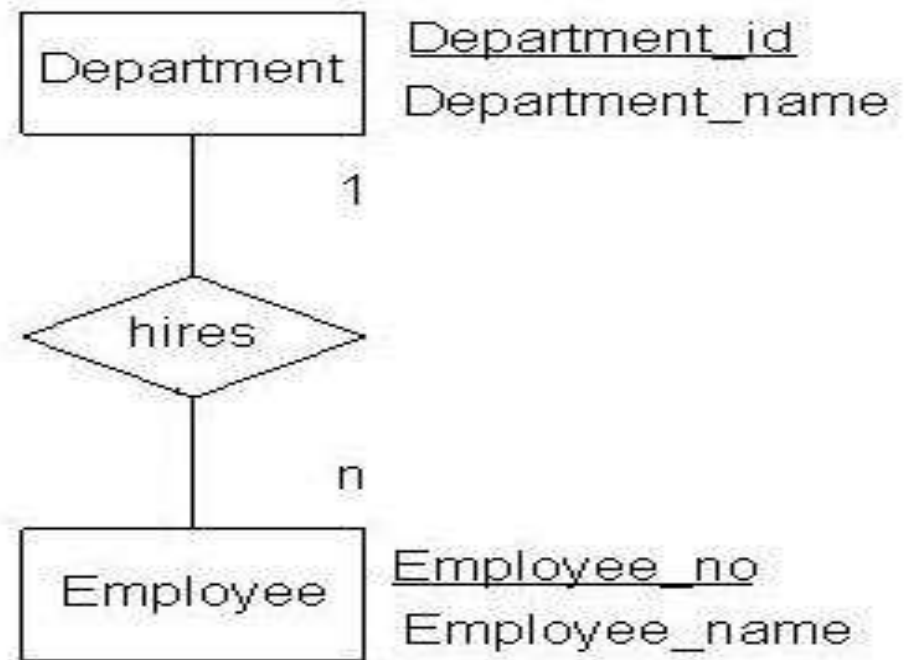
Relational Model:

Relation Department (Department_id, Department_name)

Relation Employee (Employee_no, Employee_name, &Department_id)

Where & means that null value is allowed

Extended entity relationship model:



Data Semantic: Weak Entity

The existence of a weak entity depends on its strong entity.

Example

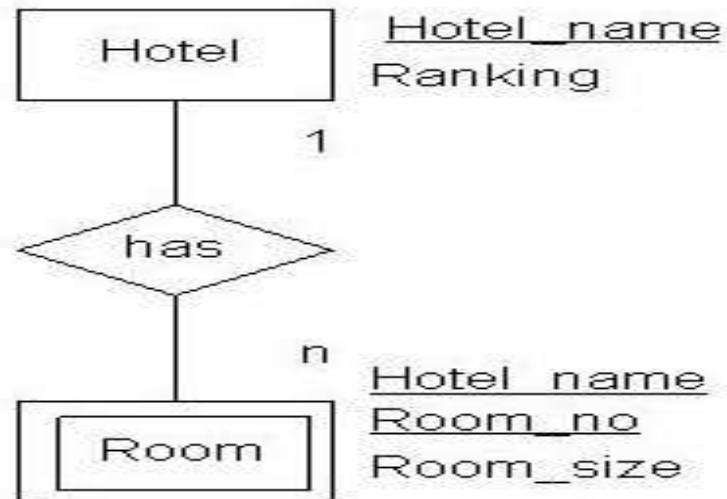
A hotel room must concatenate hotel name for identification.

Relational Model:

Relation Hotel (Hotel_name, Ranking)

Relation Room (*Hotel_name, Room_no, Room_size)

Extended entity relationship model



Cardinality: N-ary Relationship

Multiple entities relate to each other in an n-ary relationship.

Example

Employees use a wide range of different skills on each project they are associated with.

Relational Model:

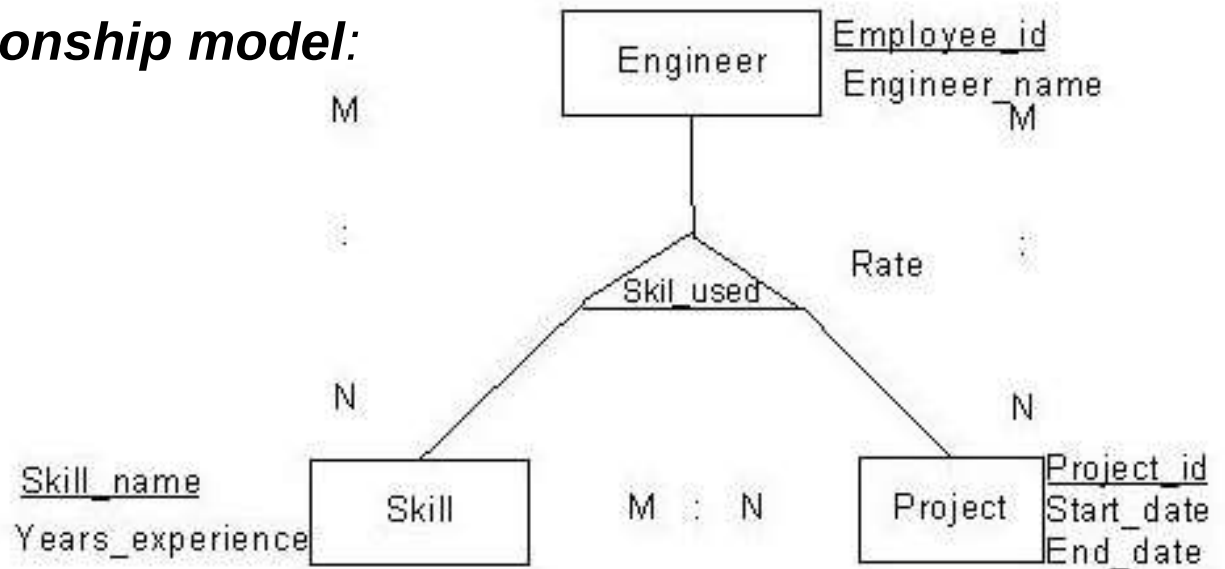
Relation Engineer (Employee_id, Employee_name)

Relation Skill (Skill_name, Years_experience)

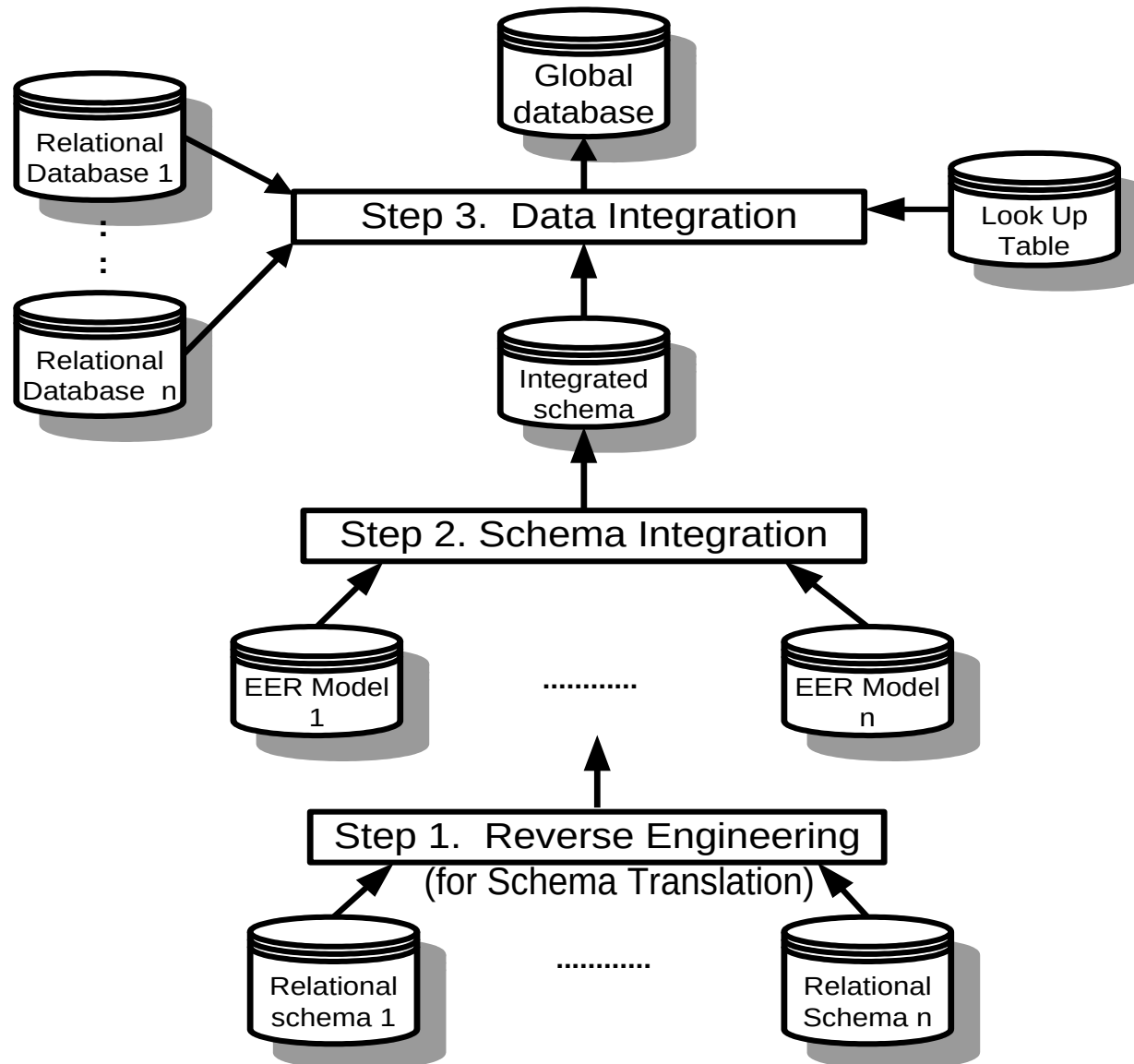
Relation Project (Project_id, Start_date, End_date)

Relation Skill_used (*Employee_id, *Skill_name, *Project_id)

Extended entity relationship model:



Architecture of multiple databases integration



Reverse engineering relational schema into EER model

Step 1 Defining each relation, key and field:

- 1) The relations are preprocessed by making any necessary candidate key substitutions as follows:*
 - *Primary relation: describing entities.*
 - *Primary relation - Type 1 (PR1). This is a relation whose primary key does not contain a key of another relation.*
 - *Primary relation - Type 2 (PR2). This is a relation whose primary key does contain a key of another relation.*

- Secondary relation: *a relation whose primary key is fully or partially formed by concatenation of primary keys of other relations.*
 - Secondary relation - Type 1 (SR1). If the key of the secondary relation is formed by concatenation of primary keys of primary relations, it is of Type 1 or SR1.
 - Secondary relation - Type 2 (SR2). Secondary relations that are not of Type 1.
- Key attribute - Primary (KAP). This is an attribute in the primary key of a relation, and is also a foreign key of another relation.
- Key attribute - General (KAG). These are all the other primary key attributes in a secondary relation that are not of the KAP type.

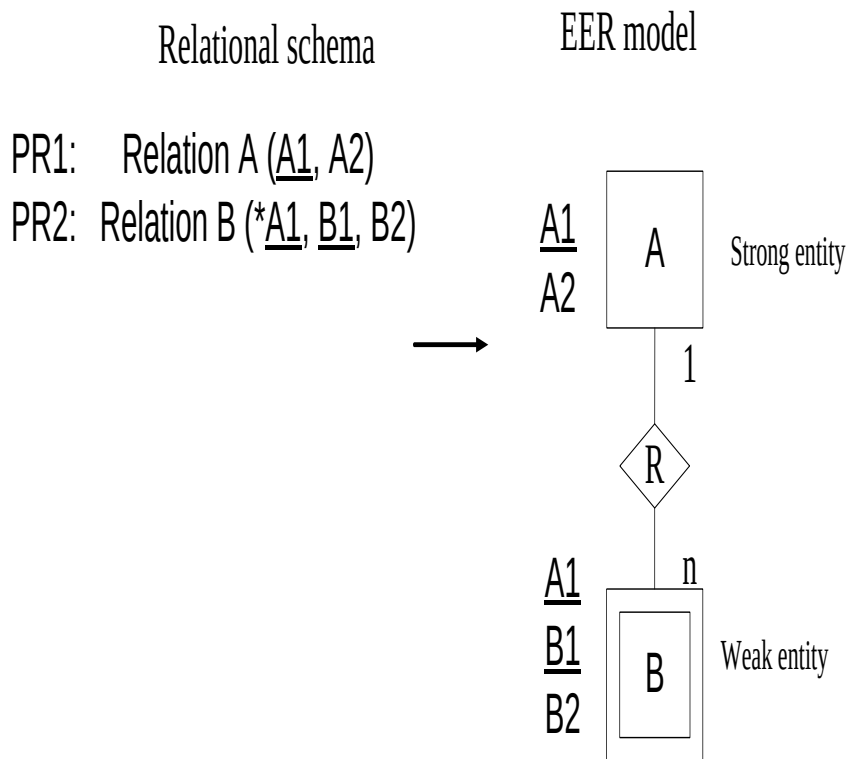
- Foreign key attribute (FKA). This is a non-primary-key attribute of a primary relation that is a foreign key.
- Nonkey attribute (NKA). The rest of the non-primary-key attribute.

Step 2 Map each PR1 into entity

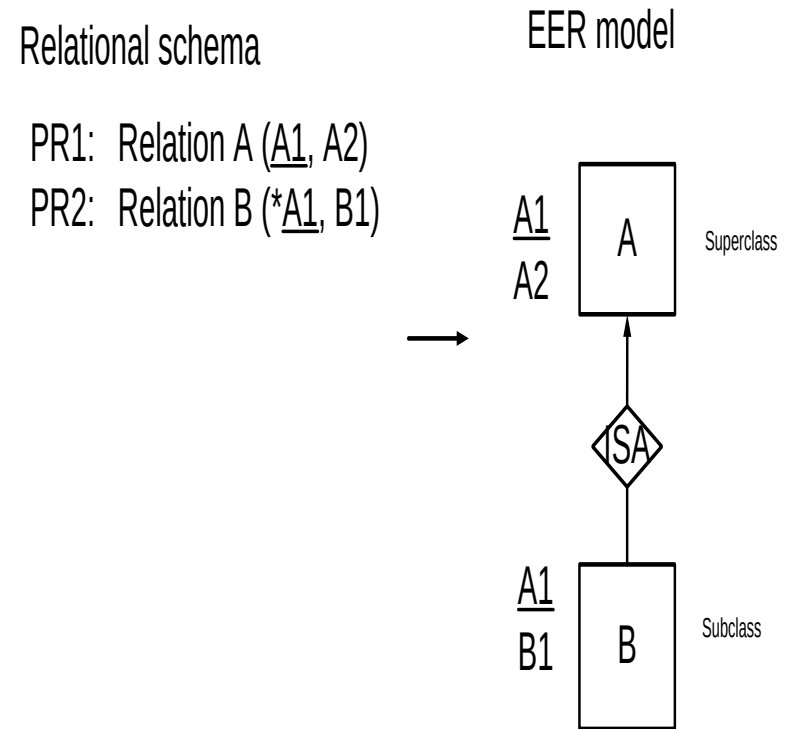
- For each Type 1 primary relation (PR1), define a corresponding entity type and identify it by the primary key. Its nonkey attributes map to the attributes of the entity types with the corresponding domains.

Step 3 Map each PR2 into a subclass entity or weak entity

Case 1:



CASE 2:



Step 4 Map SR1 into binary/n-ary relationship

Relational schema

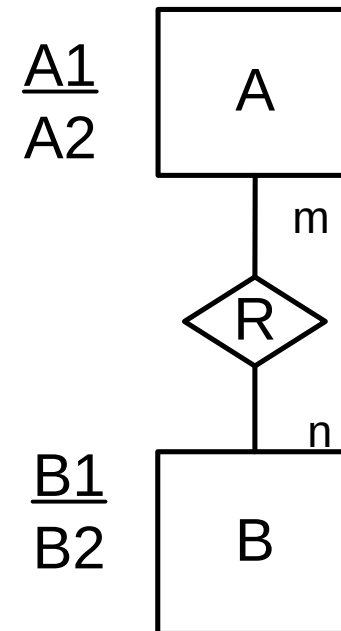
PR1: Relation A (A1, A2)

PR1: Relation B (B1, B2)

SR1: Relation AB (*A1, *B1)



EER model

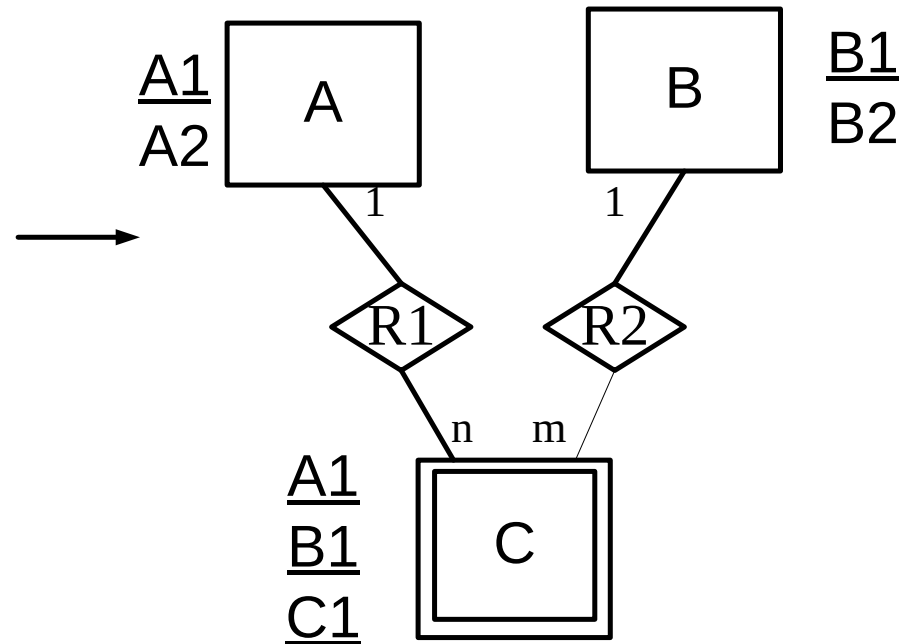


Step 5 Map SR2 into binary/n-ary relationship

Relational schema

PR1: Relation A (A1, A2)
PR1: Relation B (B1, B2)
SR2: Relation C (*A1, *B1, C1)

EER model



Step 6 Map each FKA into relationship

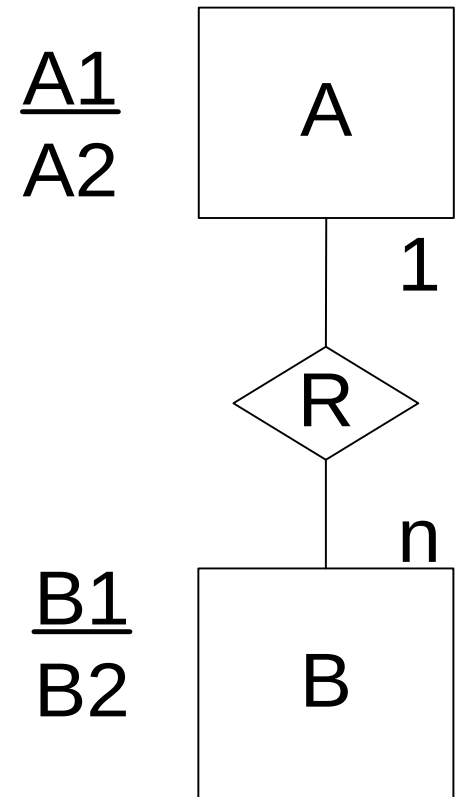
Relational schema

PR1: Relation A (A1, A2)

PR1: Relation B (B1, B2, *A1)



EER model



Step 7 Map inclusion dependency into semantic

If IDs have been derived between two relations, relation A with a as primary key and b' as foreign key, relation B with b as primary key and a' as foreign key, then

Case 1. Given ID: $a' \sqsubseteq a$, then entity A is in 1:n relationship with entity B.

Case 2. Given ID: $a' \sqsubseteq a$, and $b' \sqsubseteq b$, then entity A is in 1:1 relationship with entity B.

Case 3. Given ID: $a' \sqsubseteq a$, and $b' \sqsubseteq b$, and $a'b'$ is a composite key, then entity A is in m:n relationship with entity B.

Step 8 Draw EER model.

2026/1/21

Draw the derived EER model as a result of above steps.

An university enrollment system:

Relations:

Department (Dept#, Dept_name,)

Instructor (*Dept#, Inst_name, Inst_addr)

Course (Course#, Course_location)

Prerequisite (Prer#, Prer_title, *Course#)

Student (Student#, Student_name)

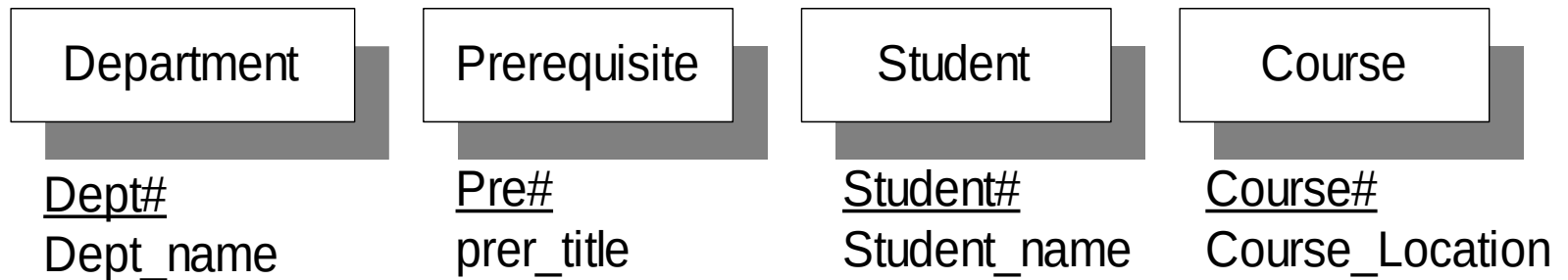
Section (*Dept#, *Course#, *Inst_name, Section#)

Grade (*Dept#, *Inst_name, *Course#, *Student#,
*Section#, Grade)

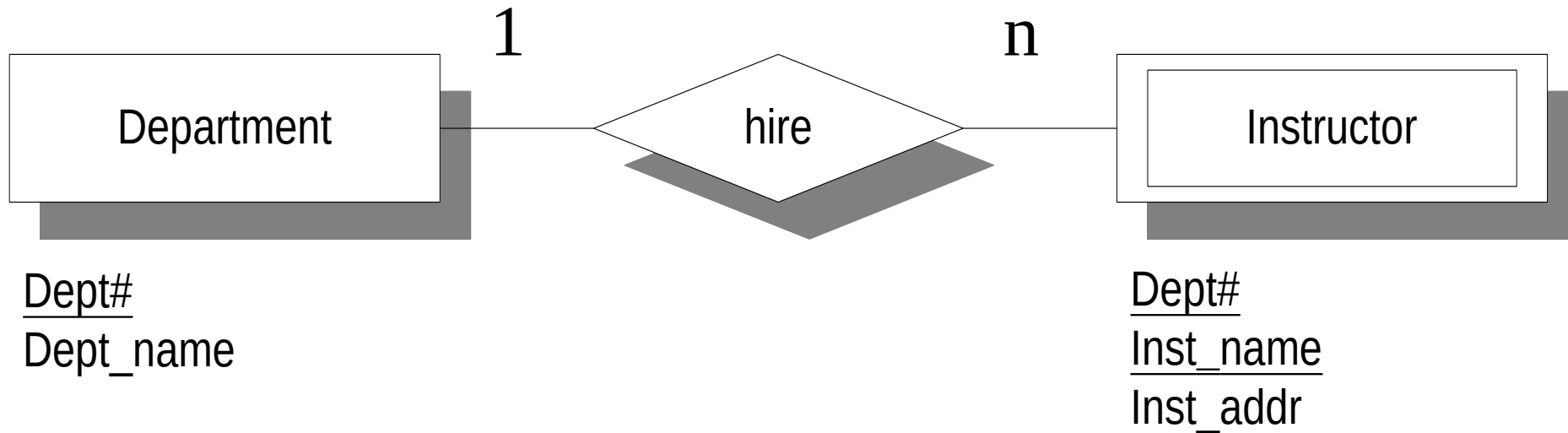
Relations and attributes classification table

Relation	Rel	Primary-	KAP	KAG	FKA	NKA
<u>Name</u>	<u>Type</u>	<u>Key_____</u>	_____	_____	_____	_____
DEPT	PR1	Dept#				Dept_name
INST	PR2	Dept#	Dept#	Inst_name		Inst_addr
		Inst_name				
COUR	PR1	Course#				Course_location
STUD	PR1	Student#				Stud_name
PREP	PR1	Prer			Course#	Prer_title
SECT	SR2	Course#	Course#	Section#		Inst_name
		Dept#	Dept#			
		Section#				
		Inst_name	Inst_name			
GRADE	SR1	Inst_name	Inst_name			Grade
		Course#	Course#			
		Student#	Student#			
		Dept#	Dept#			
		Section#	Section#			

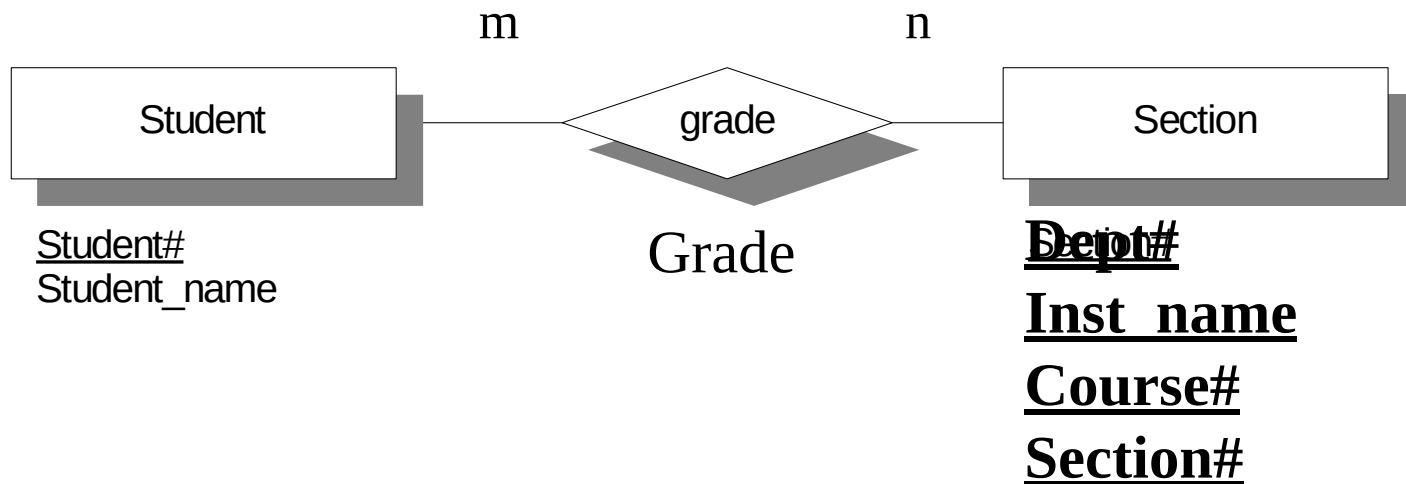
Step 2. Map each PR1 into entity



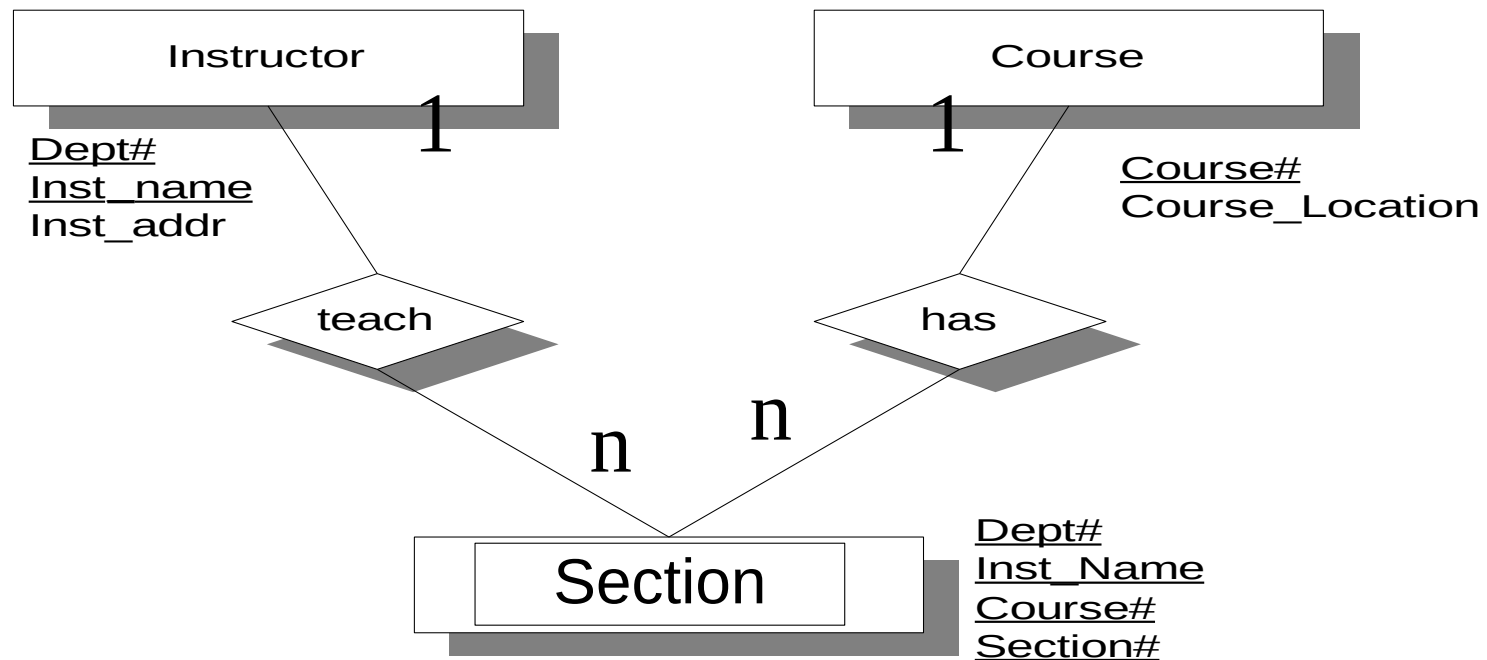
Step 3. Map each PR2 into weak entity.



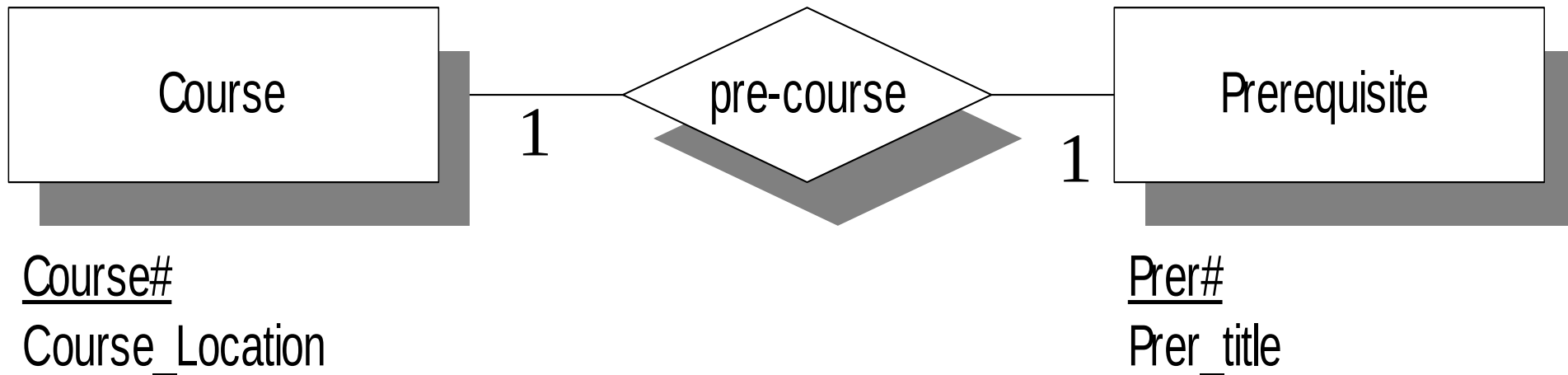
Step 4. Map SR1 into binary/n-ary relationship.



Step 5. Map SR2 into binary/n-ary relationship



Step 6. Map each FKA into relationship



Step 7. Map each inclusion dependency into semantics (binary/n-ary relationship)

Given derived inclusion dependency

$\text{Instructor.Dept\#} \subseteq \text{Department.Dept\#}$

$\text{Section.Dept\#} \subseteq \text{Department.Dept\#}$

$\text{Section.Inst_name} \subseteq \text{Instructor.Inst_name}$

$\text{Section.Course\#} \subseteq \text{Course.Course\#}$

$\text{Grade.Dept\#} \subseteq \text{Section.Dept\#}$

$\text{Grade.Inst_name} \subseteq \text{Section.Inst_name}$

$\text{Grade.Course\#} \subseteq \text{Section.Course\#}$

$\text{Grade.Student\#} \subseteq \text{Student.Student\#}$

$\text{Prerequisite.Course\#} \subseteq \text{Course.Course\#}$

$\text{Course.Prer\#} \subseteq \text{Prerequisite.Prer\#}$

Derived Semantics

n:1 relationship between entities

Instructor and Department

1:n relationship between entities

Instructor and Section

and between Course and Section.

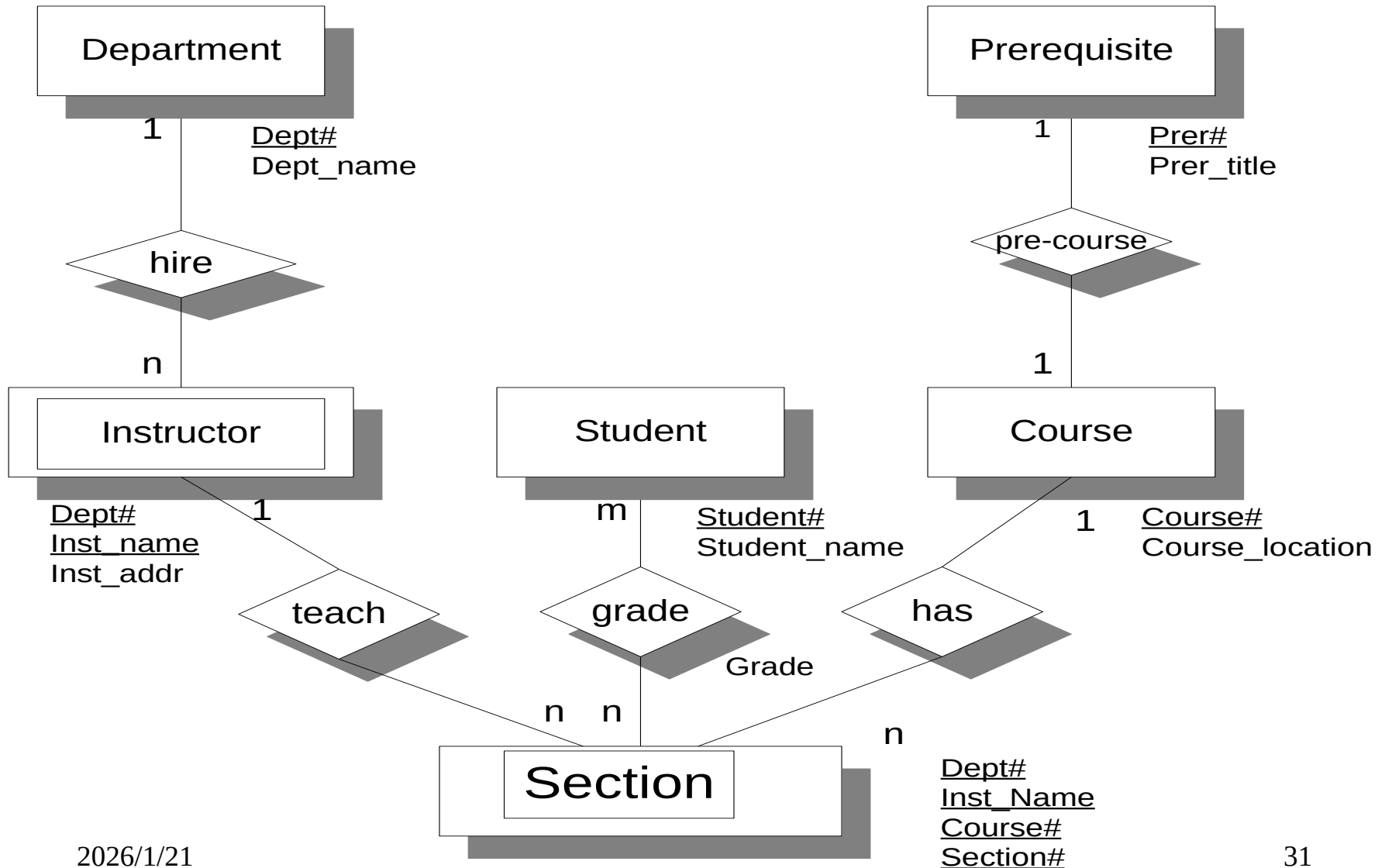
m:n relationship between

relationship Section and entity Student.

1:1 relationship between Course and

Prerequisite

Step 8. Draw *EER* model.



Reading assignment

Chapter 3 Schema Translation in “Information Systems Reengineering and Integration” by Joseph Fong, published by Springer Verlag, 2006, pp. 115-121.

Review question 2

What are the major differences between Generalization and Categorization in terms of data volume (data occurrences) in their related superclass entity/entities and subclass entity/entities?

Is there any special case such that these two data semantics can be overlapped?

Tutorial Question 2

In a reverse engineering approach, translate the following relational schema to an Entity-relationship model.

Order	(<u>Order_code</u> , Order_type, Our_reference, Order_date, Approved_date, *Head, *Supplier_code)
Supplier	(<u>Supplier_code</u> , Supplier_name)
Product	(<u>Product_code</u> , Product_description)
Department	(<u>Department_code</u> , Department_name)
Head	(<u>Head</u> , *Department_code, Title)
Order_Product	(* <u>Order_code</u> , * <u>Product_code</u> , Qty, Others, Amount)
Note	(* <u>Order_code</u> , <u>Sequence#</u> , Note)

where underlined are primary keys and prefixed with ‘*’ are foreign keys.